

Guida Build & Deploy

Sommario

Installazione e Pre-requisiti software.....	1
Configurazione Credenziali	2
Configurazione dei Database.....	2
PostgreSQL	2
MongoDB	3
Avvio del Sistema	3
Esecuzione dei Test.....	4
Procedura di Test:	4

Installazione e Pre-requisiti software

- Docker: Necessario per la containerizzazione l'orchestrazione dei servizi.
- IDE: IntelliJ IDEA (Ultimate) o PyCharm (Fortemente consigliati per l'esecuzione delle richieste HTTP tramite file *api.test.http*)

Configurazione Credenziali

Il sistema necessita di credenziali valide per autenticarsi presso *OpenSky* tramite protocollo OAuth2.

1. Scarica dal sito di OpenSky il file **credentials.json** che contengono un **ClientID** e un **Client Secret**.
2. Posizionare il file all'interno della cartella del microservizio **data_collector/**.
3. Il contenuto del file deve rispettare rigorosamente il seguente formato JSON, altrimenti il Data_Collector non funzionerà:

```
{"clientId": "giovanni07-api-client", "clientSecret": "vbHz0Vm20EQ12KtVli8KsaJ0ZRCn7Q8h"}
```

Configurazione dei Database

Il sistema è preconfigurato per l'avvio immediato tramite **docker-compose**. Di seguito sono riportati i parametri di connessione di default definiti nel file **docker-compose.yml**, utili per verificare i contenuti delle tabelle dei due rispettivi database utilizzati.

PostgreSQL

- **Container Name:** postgres_container
- **Porta Esposta:** 5432

- **Database:** user_db
- **Utente:** postgres
- **Password:** postgrespassword

MongoDB

- **Container Name:** mongo_container
- **Porta Esposta:** 27017
- **Database Dati:** data_db (creato automaticamente alla prima scrittura)
- **Database Autenticazione:** admin
- **Utente Root:** admin
- **Password Root:** adminpassword

Avvio del Sistema

Aprire un terminale nella cartella del progetto ed eseguire i comandi in sequenza riportati di seguito.

docker-compose down -v

Consigliato per rimuovere i container precedenti e per cancellare i volumi dei database garantendo un'inizializzazione pulita.

docker-compose up -build

Essenziale per compilare le immagini dei microservizi e avviare l'interno stack.

`docker ps`

Utile per verificare che tutti i container siano avviati correttamente.

Esecuzione dei Test

Il progetto non prevede un'interfaccia grafica in quanto non necessaria. La validazione delle funzionalità avviene tramite chiamate REST; nella directory principale del progetto è fornito il file *api.test.http*, che contiene tutta la lista delle richieste API utilizzate per testare la corretta esecuzione del sistema.

Procedura di Test:

1. Aprire il progetto con l'IDE consigliato (IntelliJ o PyCharm).
2. Aprire il file *api.tests.http*.
3. Eseguire le richieste sequenzialmente cliccando sull'icona "Run" (freccia verde accanto ad ogni definizione HTTP):
 - **Registrazione:** Conferma la creazione dell'utente e la generazione dell'hash bancario.
 - **At-Most-Once:** Conferma che il rinvio dello stesso RequestID non genera errori né duplicati.

- **Recupero Dati:** Verifica che il *Data Collector* abbia scaricato i voli (Arrivi/Partenze) e li restituisca correttamente.
- **Statistiche:** Valida il calcolo della media giornaliera.